

DIGITAL TWIN CATALOG

Operations Runbook

Import, re-import, publish, backup, restore and user management.

Project	Digital Twin Catalog (NARAGA 3D Catalog), reusable 3DCityDB template
Document	Runbook
Version	0.1 draft for review, 4 October 2026
Author	Fairuz Akmal Pradana
Repository	github.com/fakmalpradana/naraga-twin-db

Contents

Introduction

Import a CityGML file

Validate and mark the layer validated

Re-import a layer

Round-trip check (LOD4 and option A)

Publish a tileset

Backup and restore

Rotate the Cesium ion token

Upgrade 3DCityDB

Add a user

Introduction

Day-to-day procedures. Commands were run against the local Docker stack on 3 and 4 October 2026 unless marked *not yet run*. citydb-tool is 1.4.0 and flag names were checked with `--help`.

Import a CityGML file

```
scripts/import.sh USER=fairuz FILE=/path/oikn_lod1.gml DATASET=oikn THEME=building LOD=1
```

The wrapper does these steps and refuses to run without `USER` :

1. Creates the layer (draft) if it does not exist, and an `import_job` row with file name, SHA-256, size and user.
2. Runs `citydb import citygml --lineage oikn.building.lod1 --reason-for-update import_job:<id> --updating-person <user> -m import_all --compute-extent`.
3. Marks the job `succeeded` or `failed`, stores object counts per class, and runs `catalog.refresh_derived()` (counts and bounding boxes).

Checked on a sample: the tag `oikn.building.lod4` was written on the building **and** on its room (nested features get the lineage too), and `val_lod` held `1` and `4` for the two geometries.

Before importing real data check that gml:ids are unique across datasets, that the source CRS equals the database SRID (reproject first otherwise) and note the vertical datum on the dataset.

Validate and mark the layer validated

1. Run val3dity (or CityDoctor) on the source; `ST_IsValid` is a 2D check only. *Not yet run: no real data yet.*
2. Store the report and result, then move the layer. In the admin UI open the import job and set *Validation passed* and the report, or in SQL:

```
SELECT set_config('app.user', 'fairuz', false);
UPDATE catalog.import_job SET validation_passed = true, report = '{"tool":"val3dity"}' WHERE
id = '<job>';
UPDATE catalog.layer SET validated_by_job_id = '<job>', status = 'validated' WHERE id =
'<layer>';
```

The database refuses `validated` unless that job succeeded, passed validation and filled this layer.

Re-import a layer

Delete the old objects by lineage, then import again. Verified command (the CQL filter works; `--sql-filter` did not match anything in this version):

```
docker run --rm --platform linux/amd64 --network <compose-network> 3dcitydb/citydb-tool:1.4.0
delete \
  -H db -d postgres -u postgres -p <password> --delete-mode=delete -f "lineage =
'oikn.building.lod1'"
```

A published layer stays published after a re-import but shows **stale** until the tiles are rebuilt.

Round-trip check (LOD4 and option A)

Sample test run on 4 October 2026 with a synthetic CityGML 2.0 file (one building with `lod1Solid`, one room with `lod4Solid`, 12 polygons, 2 generic attributes, 1 name): `import`, then `citydb export citygml -v 2.0`. Counts of buildings, rooms, `lod1Solid`, `lod4Solid`, polygons, attributes and names were identical (**pass**). A real LOD4 sample from the provider is still needed to close this check. Compare counts and area or volume, not file text.

Publish a tileset

1. Convert with convertwin and upload to Cesium ion.
2. Add a `tileset` row (admin UI): provider `cesium_ion`, `ion_asset_id`, `source_snapshot_at`, `height_offset_m`, status `ready`.
3. In the tileset list choose the action *Make this build the active one*. It retires the previous build and activates this one in one step.
4. Set the layer to `published`. The database refuses it without an active Ready tileset.

Backup and restore

Nightly job `deploy/backup/backup.sh` (`pg_dump -Fc` , optional upload to an S3-compatible bucket). Local test on 4 October 2026: dump 240 KB, restored into a fresh 3DCityDB container with the same `SRID` , then `db/tests/invariants.sql` gave 19 PASS and no restore errors.

Restore procedure:

1. Start a fresh `3dcitydb/3dcitydb-pg` container with the same `SRID` and `HEIGHT_EPSG` .
2. Create the group roles first (the dump does not contain roles): `twin_ro` , `twin_editor` (in ro), `twin_admin` (in editor), `twin_importer` (in editor), and login roles `twin_app` (in admin), `twin_api` (in ro).
3. `pg_restore --clean --if-exists --no-owner -d postgres file.dump` .
4. Run `db/tests/invariants.sql` .

Rotate the Cesium ion token

Create a new read-only token for the assets in use in Cesium ion, give it to the frontend configuration, then revoke the old one. The API stores no token.

Upgrade 3DCityDB

Pin the new image tag in a staging copy, run the official database upgrade, run `dbmate up` (re-creates `feature_lineage_idx` if missing), then the tests, then promote. *Not yet run.*

Add a user

Admin UI, Users, add username and password, then choose group `viewer`, `editor` or `admin`. One account per person. Editing is allowed only when `ADMIN_EDIT_ENABLED=1`.